
1001224 - INGEGNERIA DEL SOFTWARE A - Z

Docente: **Orazio TOMARCHIO**

Risultati di apprendimento attesi

Il corso presenta i principi, i metodi e gli strumenti principali dell'Ingegneria del Software, settore dell'informatica dedicato allo studio delle metodologie, delle ricerche e degli strumenti utilizzati nella produzione industriale del software.

In particolare il corso descrive vari modelli del processo di sviluppo del software presentando i problemi relativi alle varie fasi del ciclo di vita, con particolare riferimento all'analisi ed alla specifica dei requisiti, alla definizione dell'architettura di sistema, al progetto, ed al testing. Vengono anche descritti alcuni dei più diffusi design pattern ed il loro ruolo nella progettazione e sviluppo del software. Ampio spazio viene dato al linguaggio UML, standard di riferimento per la modellazione visuale dei sistemi software.

Conoscenza e capacità di comprensione (knowledge and understanding)

- Conoscere i principi, le metodologie e gli strumenti principali nei processi di sviluppo del software, con particolare riferimento all'analisi ed alla specifica dei requisiti, alla definizione dell'architettura di sistema, al progetto, ed al testing.
- Conoscere alcuni dei più diffusi design pattern e comprendere il loro ruolo nella progettazione e sviluppo del software.
- Conoscere la notazione standard UML per la modellazione dei sistemi software.

Capacità di applicare conoscenza e comprensione (applying knowledge and understanding)

- Saper progettare un sistema informatico con architettura mediamente complessa, pianificando le varie attività dei processi del ciclo di vita del software e producendo documenti in accordo agli standard del settore.
- Saper modellare le diverse viste di un sistema software utilizzando la notazione standard UML.
- Saper riconoscere quando e quali design pattern applicare durante la progettazione di un sistema software.

Autonomia di giudizio (making judgements)

- Lo studente sarà in grado di valutare le diverse scelte progettuali nel corso dello sviluppo di un sistema informatico, identificando le soluzioni più indicate nel rispetto dei requisiti e degli interessi manifestati dagli stakeholders

Abilità comunicative (communication skills)

- Lo studente acquisisce la conoscenza delle notazioni standard per la modellazione di un sistema software, che lo rendono in grado di interagire e comunicare al meglio all'interno di un team di sviluppo software

Capacità di apprendere (learning skills)

- Lo studente è in grado di apprendere ed applicare nuovi modelli di sviluppo del software che si basino sulle fasi fondamentali analizzate durante il corso

Modalità di svolgimento dell'insegnamento

Il corso prevede come metodo di insegnamento:

- le lezioni frontali per acquisire le conoscenze teoriche e metodologiche della disciplina
- lo svolgimento di esercitazioni proposte dal docente per acquisire la capacità di risolvere i problemi, applicare la conoscenza e le metodologie di progettazione studiate e utilizzare degli ambienti di sviluppo software

Il docente propone, inoltre, delle esercitazioni (homework) che consistono nella soluzione di un problema progettuale che gli studenti devono affrontare in gruppi in modo autonomo, che vengono successivamente corrette e/o discusse in aula.

Qualora l'insegnamento venisse impartito in modalità mista o a distanza potranno essere introdotte le necessarie variazioni rispetto a quanto dichiarato in precedenza, al fine di rispettare il programma previsto e riportato nel syllabus.

Prerequisiti richiesti

- Fondamenti di programmazione
- Programmazione orientata agli oggetti

Frequenza lezioni

Frequenza non obbligatoria ma fortemente consigliata

Contenuti del corso

1. Concetti e definizioni di base dell'Ingegneria del Software
 - Introduzione. Origini e motivazioni dell'Ingegneria del Software. Definizioni di base: prodotti software, caratteristiche generali dei prodotti software. Ciclo di vita del software. Processi per lo sviluppo del software: modello a cascata, sviluppo incrementale; modello iterativo/evolutivo, modello prototipale, modello a spirale, unified process.
2. Analisi e specifica dei requisiti
 - Definizione del concetto di requisito. Requisiti funzionali e non funzionali. Attività di definizione, analisi e specifica dei requisiti. Processo di ingegnerizzazione dei requisiti. Documentazione dei requisiti. Validazione dei requisiti.
3. Progettazione software
 - Progetto del software. Metodi di progetto: approccio top-down, metodi strutturati, strategie funzionali e object oriented. Documentazione del progetto. Parametri di qualità del progetto: coesione, accoppiamento, comprensibilità e adattabilità. Progetto dell'architettura logica. Modelli per la strutturazione dei sistemi software (pattern architetturali). Principi di analisi e progettazione orientata agli oggetti.
4. La modellazione del software con UML
 - Generalità su UML (Unified Modeling Language). UML e ciclo di vita. Modellare i requisiti con i casi d'uso. Diagrammi delle classi e degli oggetti. Diagrammi di sequenza e collaborazione. Diagramma degli stati. Diagramma di attività. Diagramma dei componenti e di deployment. Strumenti CASE a supporto di UML.
5. Design pattern per la progettazione ed il riuso
 - Ruolo dei design pattern nella progettazione e sviluppo del software. Pattern GRASP. Pattern GoF: creazionali, strutturali, comportamentali.

6. Verifica e validazione

- Il controllo di qualità dei prodotti software: la verifica e la validazione. Verifica e validazione statiche e dinamiche. Testing e ispezione. Obiettivi e problematiche generali del testing. Pianificazione e organizzazione dei test. Strategie di test. Test dinamico black box (funzionale) e white box (strutturale). Il concetto di test case. Dati di test. Classi di equivalenza. Testing dei cammini, grafi di flusso, complessità ciclotomica. JUnit.

7. Software development management

- Software configuration management. Configuration item, version, configurazioni, repository. Utilizzo di tool di versioning (CVS, SVN, Git). Gestione delle build, release e branch.

Testi di riferimento

- [LAR]
 - Craig Larman
Applicare UML e i pattern – Analisi e progettazione orientata agli oggetti
Pearson Education Italia
- [FOW]
 - M. Fowler
UML Distilled
Pearson Education Italia
- [GAM]
 - Gamma, E., Helm, R., Johnson, R. e Vlissides, J.
Design Patterns: elementi per il riuso di software a oggetti
Addison Wesley
- [SOM] (*Testo di consultazione per alcuni argomenti*)
 - I. Sommerville
Ingegneria del Software (8 edizione)
Pearson Education Italia
- Appunti e dispense fornite dal docente su alcuni argomenti (pubblicate sul canale MS Teams del corso: codice okl3645)

Programmazione del corso

Argomenti		Riferimenti testi
1	Introduzione all'Ingegneria del Software	[SOM] Cap. 1
2	Overview Analisi e Progettazione orientata agli oggetti	[LAR] Cap. 1
3	Ciclo di vita e Processi di sviluppo del Software	[LAR] Cap. 2 e 3 - [FOW] Cap. 2
4	La modellazione del software. Introduzione a UML	[FOW] Cap. 1
5	Generalità sulla fase di ideazione	[LAR] Cap. 4 e 5
6	Requisiti	[LAR] Cap. 6 - [SOM] Cap. 6
7	Casi d'uso	[LAR] Cap. 7 - [FOW] Cap. 9
8	Ulteriori Elaborati sui requisiti	[LAR] Cap. 8

Argomenti		Riferimenti testi
9	La fase di elaborazione	[LAR] Cap. 10-11
10	Modelli di Dominio	[LAR] Cap. 12 - [FOW] Cap. 3
11	Diagrammi di sequenza di sistema	[LAR] Cap. 13
12	Contratti delle Operazioni	[LAR] Cap. 14
13	Verso la Progettazione. Cenni su Architetture Software	[LAR] Cap. 15-16-17
14	Diagrammi di Interazione in UML	[LAR] Cap. 18 - [FOW] Cap. 4-12
15	Diagrammi delle Classi in UML	[LAR] Cap. 19 - [FOW] Cap. 3-5
16	Pattern GRASP: progettazione di oggetti con responsabilità	[LAR] Cap. 20
17	Esempi di progettazione di oggetti con i pattern GRASP	[LAR] Cap. 21
18	Trasformare i progetti in codice	[LAR] Cap. 22 - 23
19	Dall'iterazione 1 all'iterazione 2	[LAR] Cap. 26 - 27
20	Ulteriori pattern GRASP	[LAR] Cap. 28
21	Raffinamento Modello di Dominio	[LAR] Cap. 34 - [FOW] Cap. 5
22	Diagrammi di attività in UML	[LAR] Cap. 31 - [FOW] Cap. 11
23	Diagrammi di macchina a stati in UML	[LAR] Cap. 32 - [FOW] Cap. 10
24	Introduzione ai Design Pattern GoF	[GAM] Cap. 1
25	Pattern creazionali	[GAM] Cap. 3
26	Pattern strutturali	[GAM] Cap. 4
27	Pattern comportamentali	[GAM] Cap. 5
28	Esempi sui pattern	[LAR] Cap. 29
29	Correlare i casi d'uso. Use Case Diagram in UML	[LAR] Cap. 33 - [FOW] Cap. 9
30	Diagrammi di deployment e dei componenti in UML	[LAR] Cap. 41 - [FOW] Cap. 8-14
31	Introduzione al testing del Software	Dispense del docente - [SOM] Cap. 22
32	Criteri di testing	Dispense del docente - [SOM] Cap. 23
33	Automazione dei test	Dispense del docente

Argomenti		Riferimenti testi
34	JUnit	Dispense del docente
35	Gestione della configurazione del software. Sistemi di controllo delle versioni	Dispense del docente
36	SVN	Dispense del docente
37	Git	Dispense del docente
38	Evoluzione dei sistemi distribuiti e tecnologie di middleware	Dispense del docente

Verifica dell'apprendimento

Modalità di verifica dell'apprendimento

L'esame consiste in:

- sviluppo di un elaborato (progettazione di un sistema software utilizzando le metodologie, le tecniche e gli strumenti visti studiati durante il corso)
- prova orale (discussione progetto + eventuali domande inerenti parti del programma non coperte dal progetto.)

Valutazione esame complessivo: Elaborato 70% - Prova orale 30%

La verifica dell'apprendimento potrà essere effettuata anche per via telematica, qualora le condizioni lo dovessero richiedere.

Esempi di domande e/o esercizi frequenti

Esercizi tipici ed esempi di elaborati sono presenti sul portale di Ateneo studium.unict.it e/o la piattaforma Microsoft Teams

English version